

## 05 — Migliorare per tentativi: la salita di collina

**Sessione di studio.** Prima il meccanismo in astratto, poi lo stesso meccanismo nel codice del progetto (`src/engine/bilancia.js`, `src/engine/forza.js`), infine le domande di verifica.

### 1. Il problema: quando non esiste una formula

Fin qui, nel progetto, quasi tutto si è calcolato: il tipo di un'energia si **deduce** dal nome, le linee evolutive si **ricostruiscono** dai dati, la lista di un mazzo si **stampa** da un array. Domanda in, risposta out.

«Fammi due mazzi di pari forza» non è di questa famiglia. E nemmeno «fammi mazzi di forza 45». Non esiste una formula che, data una scatola di carte, restituisca la combinazione giusta: esiste solo un modo di **misurare** quanto una combinazione è buona, e un numero enorme di combinazioni da provare. Con 100 carte diverse e mazzi da 15, le combinazioni possibili sono più delle stelle della galassia: enumerarle tutte non è «lento», è impossibile.

Questa è la forma classica di un **problema di ottimizzazione combinatoria**, e si affronta con la **ricerca locale**: invece di cercare *la* soluzione, si parte da una soluzione qualsiasi e la si **migliora a piccoli passi**.

Servono tre ingredienti, e conviene saperli nominare perché li ritroverai identici in qualunque problema di questo tipo:

| Ingrediente                     | Cos'è                                                         | Nel progetto                                              |
|---------------------------------|---------------------------------------------------------------|-----------------------------------------------------------|
| <b>Funzione obiettivo</b>       | un numero che dice quanto è buona una soluzione               | <code>punteggioMazzo()</code> in <code>bilancia.js</code> |
| <b>Vicinato</b>                 | le mosse elementari che trasformano una soluzione in un'altra | scambiare una carta con una libera in collezione          |
| <b>Criterio di accettazione</b> | quando una mossa si tiene e quando si scarta                  | solo se il punteggio si avvicina a quel che si vuole      |

La **salita di collina** (*hill climbing*) è la strategia più semplice che li usa tutti e tre: guarda le mosse possibili, fai la migliore, ripeti finché nessuna mossa migliora. Il nome viene dall'immagine: sei su una collina nella nebbia, fai un passo nella direzione in cui il terreno sale, e ti fermi quando intorno a te tutto scende.

### 2. La funzione obiettivo è una scelta, non una verità

Questo è il punto che si sottovaluta sempre. L'algoritmo **ottimizzerà esattamente ciò che misuri**, comprese le stupidaggini che misuri per sbaglio.

`punteggioMazzo()` misura quattro cose, con dei pesi:

```
// src/engine/bilancia.js
totale: Math.round(ps * 0.6 + danno * 2 + evoluzione * 12 + coerenza * 20);
```

Nota due decisioni dentro la formula, e come sono argomentate nel codice:

- il danno non è quello assoluto ma **per energia spesa** — «un attacco da 120 che ne costa quattro è più debole di uno da 40 che ne costa una, in una partita corta»;
- i gradini evolutivi contano solo se **giocabili** — un Livello 2 senza la sua linea nel mazzo «è una carta morta, non una carta forte».

Se il secondo punto non ci fosse, la salita di collina imparerebbe subito a riempire i mazzi di evoluzioni orfane: sono le carte con più PS e più danno, e il punteggio salirebbe magnificamente. Otterresti un mazzo con un punteggio altissimo e ingiocabile. **Un difetto della funzione obiettivo non produce un errore: produce una soluzione perfetta al problema sbagliato.**

**Rispetto a Java.** È la stessa disciplina di un Comparator: definisce *cosa vuol dire* «maggiore», e tutto ciò che ordini dopo ne dipende. Solo che qui un Comparator sbagliato non ti dà una lista disordinata — ti dà una partita rovinata sabato pomeriggio.

### 3. Due obiettivi diversi, lo stesso schema

Il progetto usa la salita di collina in **due** punti, ed è istruttivo vederli affiancati perché l'impalcatura è identica e cambia solo cosa si insegue.

bilancia.js — «i mazzi devono somigliarsi»

L'obiettivo è la **differenza** fra il mazzo più forte e il più debole: `squilibrio()` la calcola, e si cerca di **mini-mizzarla**. La mossa elementare non è scambiare una carta ma spostare una **linea evolutiva intera** da un mazzo all'altro:

```
// si provano TUTTE le linee e si tiene quella che pareggia di più
for (const linea of candidate) {
  const annulla = spostaLinea(ricco, povero, linea);
  const differenza = squilibrio(mazzi).differenza;
  annulla(); // ← prova annullata
  if (!migliore || differenza < migliore.differenza) migliore = { linea, differenza };
}
if (!migliore || migliore.differenza >= prima.differenza) break; // nessuna mossa migliora
```

forza.js — «i mazzi devono valere 45»

L'obiettivo è la **distanza dal numero chiesto**, `Math.abs(forza - obiettivo)`, e si cerca di minimizzare quella. La mossa elementare è scambiare **una copia** di una carta del mazzo con una copia libera in collezione:

```
// src/engine/forza.js
const annulla = scambia(mazzo, voce, carta);
if (!annulla) continue;
const forza = punteggioMazzo(mazzo).totale;
annulla();

const scarto = Math.abs(forza - obiettivo);
if (scarto < scartoMigliore) { scartoMigliore = scarto; migliore = { cartaFuori, carta, forza }; }
```

Stesso schema: **prova** □ **misura** □ **annulla**, e alla fine si applica solo la mossa migliore. Il dettaglio da rubare è proprio l'`annulla()` restituito da chi ha fatto la modifica: chi modifica sa anche come disfare, e chi prova non deve saperlo. È più solido di «faccio una copia profonda di tutto prima di ogni prova», che su decine di migliaia di prove costerebbe caro.

Che il minimo cercato sia «differenza zero» o «distanza da 45 zero» non cambia nulla per l'algoritmo. Cambia tutto per chi gioca.

### 4. I vincoli: dove l'algoritmo bara se glielo lasci fare

Una salita di collina senza vincoli trova sempre la scorciatoia. Vuoi far scendere la forza di un mazzo? La mossa più efficiente è togliere il Charizard e metterci un Pokémon di un tipo che il mazzo non alimenta: il punteggio crolla (la coerenza va a zero) e l'obiettivo è centrato. Peccato che in mano ti ritrovi una carta che non può attaccare.

Per questo `forza.js` restringe il vicinato **prima** di misurare:

```
// esce solo un Pokémon "sciolto": nessuno nel mazzo evolve da lui,
// e lui non evolve da una carta presente
function sostituibile(voce, mazzo) { ... }
```

```
// entra solo un Base di un tipo che il mazzo alimenta già (o senza tipo)
function tipoCompatibile(carta, mazzo) { ... }
```

Regola generale: **i vincoli si mettono nel vicinato, non nel punteggio**. Si può essere tentati di lasciare libere le mosse e penalizzare nel punteggio le soluzioni brutte — ma così l'algoritmo continua a esplorare un mare di soluzioni inutili, e ogni penalità va pesata contro le altre. Vietare la mossa è più semplice, più veloce e più facile da spiegare a chi legge il codice.

## 5. Quando ci si ferma, e perché va detto

La salita di collina ha un difetto noto: si ferma sul **massimo locale**, cioè sulla prima cima che incontra, che non è necessariamente la più alta. E può fermarsi anche perché le hai dato un tetto di tentativi.

Sono tre finali diversi, e confonderli significa mentire all'utente. Per questo `avvicinaUno()` non restituisce solo il numero raggiunto:

```
let motivo = 'passi'; // tetto di tentativi esaurito
if (Math.abs(corrente - obiettivo) <= tolleranza) { motivo = 'obiettivo'; break; }
const scelta = migliorScambio(...);
if (!scelta) { motivo = 'collezione'; break; } // nessuna mossa migliora più
```

e la UI dice cose diverse nei due casi in cui l'obiettivo non è stato raggiunto: «con le carte che hai non si va oltre» oppure «il motore si è fermato dopo un certo numero di scambi: prova a rigenerare». Il primo messaggio chiude il discorso, il secondo suggerisce una mossa. Dirli a caso è peggio che non dire niente.

C'è anche una **tolleranza** (`TOLLERANZA_FORZA = 5`): sotto i cinque punti la differenza non si sente giocando, e inseguirla produrrebbe scambi che nessuno noterebbe. Sapere quando smettere di ottimizzare fa parte dell'ottimizzazione.

**Se ti incuriosisce.** Il modo standard di uscire dai massimi locali è accettare ogni tanto una mossa che *peggiora*, con probabilità decrescente: si chiama *simulated annealing*. Qui non serve — il vicinato è piccolo e il risultato deve essere ripetibile — ma è il passo successivo naturale.

## 6. Purezza: perché tutto questo sta in `src/engine/`

`forza.js` non importa nulla dal DOM né da IndexedDB. Riceve i mazzi e la dispensa, li modifica, restituisce il resoconto. È la stessa regola che vale per tutto `src/engine/`, e qui si vede bene **perché** conviene: una salita di collina è codice che si sbaglia facilmente e in modo silenzioso — un annullamento che non annulla del tutto, un riferimento che punta a un oggetto sostituito — e l'unico modo di accorgersene è provarla in isolamento, con mazzi finti costruiti apposta.

Un esempio vero da `tests/forza.test.js`:

```
// La taglia è l'unica cosa che non può cambiare: un mazzo da 15 che ne conta
// 13 non si gioca, e sarebbe il modo più facile di far scendere il punteggio.
assert.equal(m.totale, carteIniziali);
```

Quel test non verifica un requisito estetico: verifica che l'ottimizzatore non abbia trovato l'ennesima scorciatoia.

E un bug che il test ha stanato davvero: il vicinato annotava le **voci** del mazzo, ma provando uno scambio l'ultima copia di una voce la fa sparire da `mazzo.carte`, e annullando rientra come voce *nuova*. Tenendo il riferimento vecchio, tutte le prove successive su quella carta fallivano in silenzio — e le carte in copia unica non venivano mai considerate. Nessun errore a video: solo mazzi ottimizzati peggio. Da lì la scelta di annotare le **carte** e ritrovare la voce a ogni prova (`vocePer()`).

## 7. Verifica

1. Nella formula di `punteggioMazzo()`, evoluzione pesa 12 e ps pesa 0,6. Cosa succederebbe ai mazzi generati se invertissi i due pesi? Prova a prevederlo **prima** di provarlo davvero cambiando il numero.

2. `bilancia()` tiene un Set di linee già spostate e non ne muove due volte la stessa. Che comportamento si otterrebbe togliendolo? (Il commento nel codice racconta cosa succedeva davvero.)
3. In `forza.js` il criterio di accettazione è `scarto < scartoMigliore`, con `<` stretto. Cosa cambierebbe con `<=`? Pensa a due scambi che lasciano lo stesso identico scarto.
4. **Esercizio.** Aggiungi a `punteggioMazzo()` una quinta voce: quante carte del mazzo si possono giocare al **primo turno** (Pokémon Base in mano). Scegli un peso, scrivi il test che ne dimostra l'effetto, e osserva come cambiano i mazzi generati a parità di collezione.
5. **Esercizio.** `avvicinaAForza()` lavora un mazzo per volta, in ordine, e i mazzi condividono la dispensa. Il primo mazzo può quindi prendersi le carte deboli e lasciare il secondo lontano dall'obiettivo. Scrivi il test che mostra il problema, poi proponi (a parole) una strategia migliore: a turno? partendo dal mazzo più lontano dall'obiettivo?
6. Perché `scambia()` restituisce una funzione invece di, per esempio, un oggetto `{voce, carta}` da rimettere a posto a mano dal chiamante? Cosa guadagna chi chiama, e cosa non è più costretto a sapere?